

數字經濟核心科技系列：前端網站開發人員課程

(二) 進階網絡程式設計

3. React JS

15. 專業ReactJS應用I

數字經濟核心科技系列：前端網站開發人員課程

(二) 進階網絡程式設計

3. React JS

ReactJS在前端網站開發的應用

數字經濟核心科技系列：前端網站開發人員課程

(二) 進階網絡程式設計

3. React JS

15.1 網絡框架應用簡介

15. 專業ReactJS應用I

課程結構回顧



All Rights Reserved. Reprinting will not be tolerated.
版權所有 · 翻印必究

15. 專業ReactJS應用I

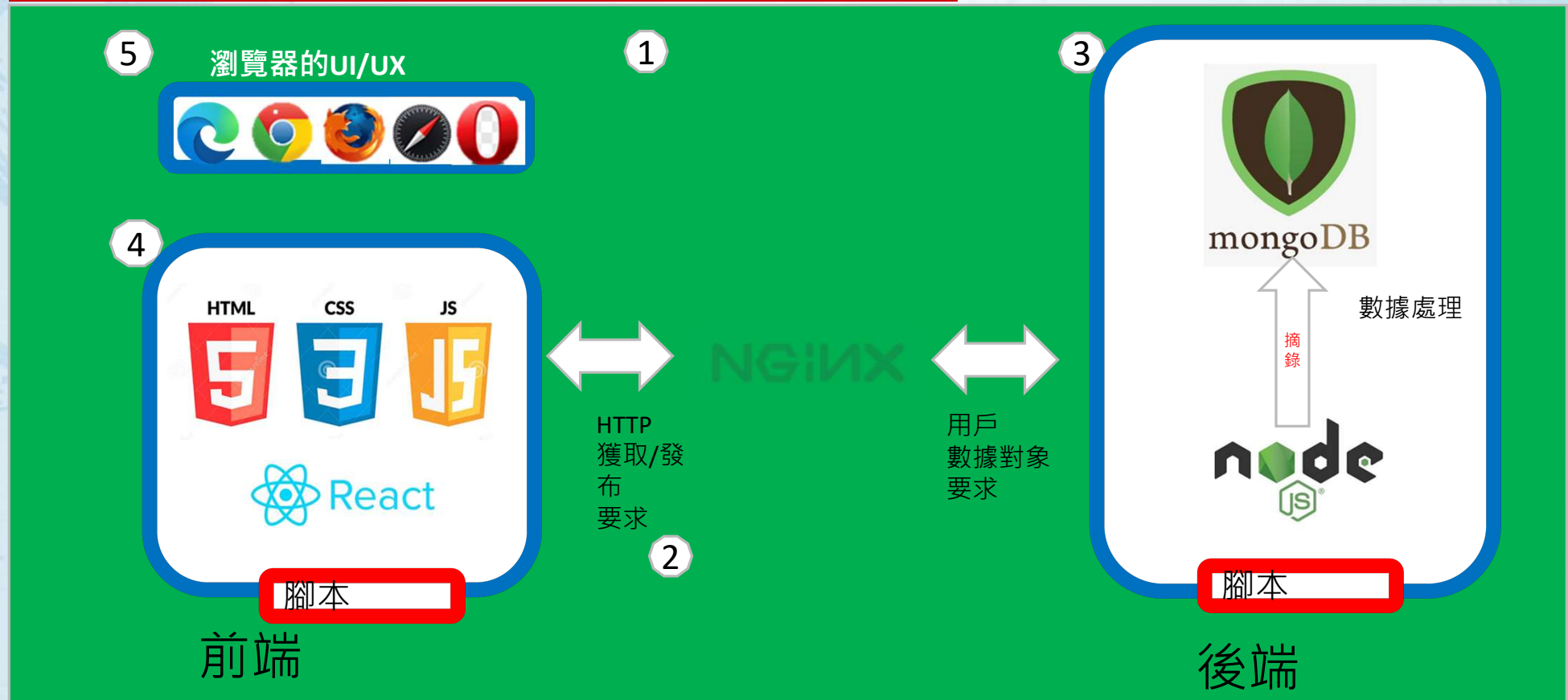
網絡應用程式結構回顧(1)



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

15. 專業ReactJS應用I

網絡應用程式結構回顧(2)



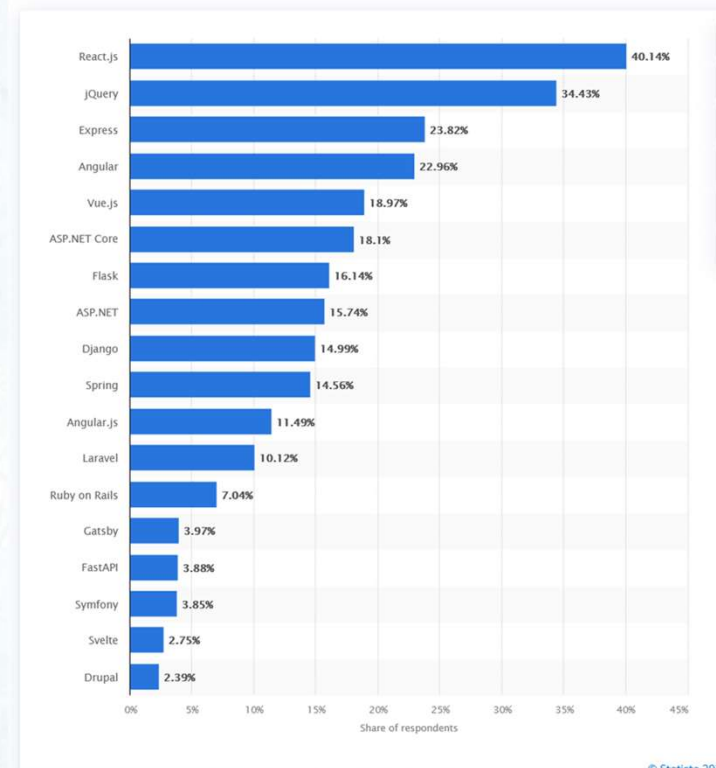
All Rights Reserved. Reprinting will not be tolerated.
版權所有 · 翻印必究

15. 專業ReactJS應用I

使用網絡框架

- 使用網絡框架(web frameworks)來提供卓越的用戶體驗並縮短開發時間
- 框架包含基本的軟件模塊，有現成的程式碼，開發人員用來解決常見的編程任務，如管理AJAX請求或定義文件結構
- 網絡框架還規定了構建應用程式架構的規則：善用程式碼骨架，根據指定的要求進行擴展和改變

Most used web frameworks among developers worldwide,



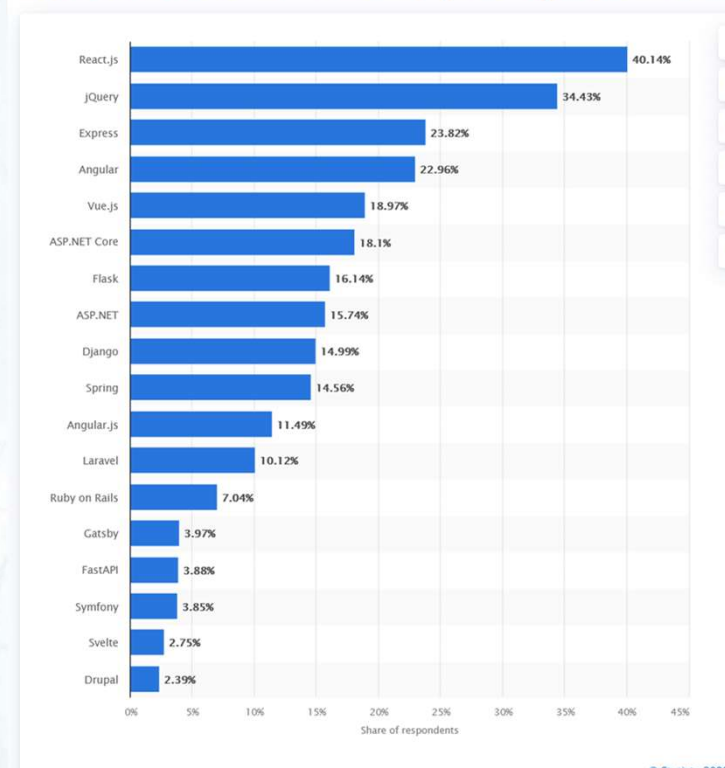
All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

15. 專業ReactJS應用I

使用網絡框架

- 框架包括實用程式(utility programs)、程式碼庫(code libraries)、腳本語言(scripting languages)和其他軟件，以促進大型軟件產品的各種組件的開發和集成
- 因為應用框架，有了實現其他項目特定功能的基礎
- 這有助於加快開發速度，提高解決方案的生產力和可靠性

Most used web frameworks among developers worldwide,

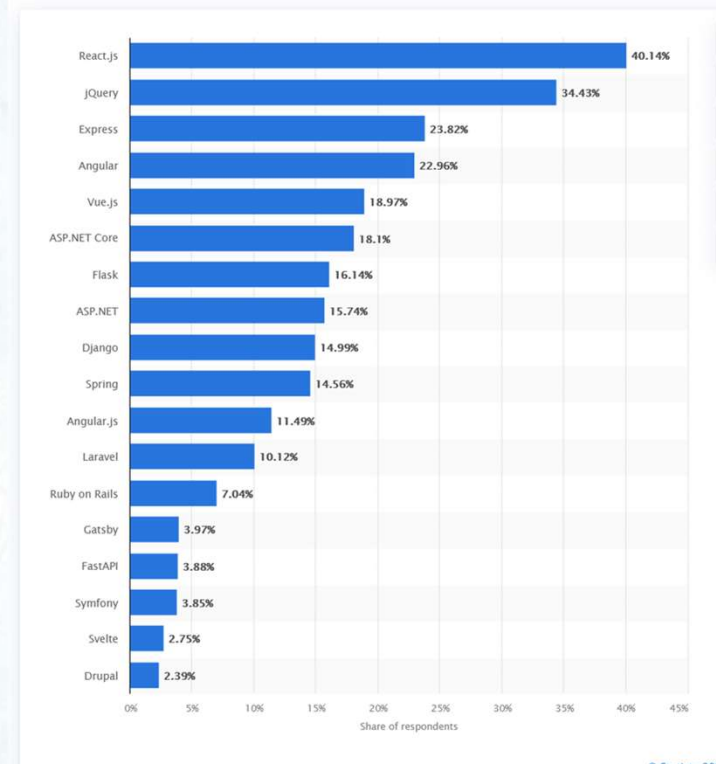


All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

15. 專業ReactJS應用I 使用網絡框架

- 網絡開發中的前端是一個網站或一個應用程式的用戶界面
- 這是用戶可以看到並與之互動的部分
- 這一部分需要對用戶友好，以有效地呈現來自後台的信息

Most used web frameworks among developers worldwide,



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

15. 專業ReactJS應用I 使用網絡框架

以下是五款最常用的網絡框架，課程將重點放在其中的兩款框架上，分別是 React JS和Node JS



15. 專業ReactJS應用I

ReactJS and JQuery

React JS：由Facebook建立，其虛擬DOM有別於其他框架對於預計會有高流量並需要一個穩定的平台，這是一個理想的框架



JQuery：最簡單的框架之一。只需要少量簡單的編程語言，也可以輕鬆使用 JQuery



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

15. 專業ReactJS應用I

NodeJS and AngularJS

Node JS：是一個快速、無偏見、簡約的網絡框架。作為一個非同步事件驅動的JavaScript執行環境，Node JS被設計用來構建可擴展的網絡應用程式



Angular JS：由Google於2010年開發的強大開源前端開發框架。具備卓越的效能與實用功能，包括指令、過濾器、雙向資料綁定、相依性等功能



15. 專業ReactJS應用I

VueJS

Vue JS：首次發佈於2015年，最新版本Vue 3.0則於2020年推出。Vue 3.0在敏捷開發方面帶來更多優勢：使軟件包變得更小，推出解決大型任務的API及支援伺服器端渲染(SSR)



15. 專業ReactJS應用I

Low code開發平台

- 低程式碼(Low code)開發平台是一種提供圖形使用者介面進行程式設計的應用程式，能以非常快的速度開發程式碼，並減少傳統程式設計的工作量



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

15. 專業ReactJS應用I

Low code開發平台

- 這些工具透過減少手工編碼工作來幫助快速開發程式碼
- 這些平台不僅協助編碼，也促進快速設置和部署

<https://budibase.com/>

<https://www.joget.org/>

<https://www.convertigo.com/>

<https://www.bswen.com/2021/11/trending-low-code-platforms-in-2022.html>

<https://github.com/huginn/huginn>

<https://thebusinessblocks.com/open-source-software/open-source-low-code-platform/>

<https://stackstorm.com/>

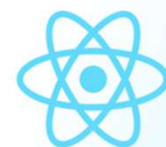


All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

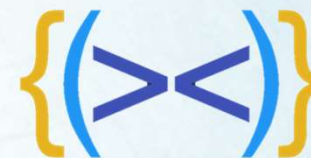
15. 專業ReactJS應用I

選擇合適的技術

- 在科技演進的時代，網絡開發是大多數企業的支柱
- 為項目決定正確的技術和相關的開發人員，有助於加速企業擴展營運，並決定產品的靈活性、可擴展性、互通性和未來發展



ReactJS

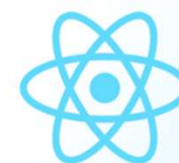


All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

15. 專業ReactJS應用I

應用ReactJS

- **React**是一個建基於組件的JavaScript庫，用於開發互動式使用者介面(UI)和建構可重複使用的UI組件
- **React**是一個非常流行的JavaScript前端庫，建立在強大基礎和龐大社群支持之上



ReactJS

15. 專業ReactJS應用I

應用ReactJS

React除了提供可重複使用的庫程式碼外，還提供兩項支援JavaScript開發的主要功能

- **JSX** - 用來操作DOM(文件對像模型)
- **虛擬DOM** - 具有與真實DOM相同的屬性，能大幅提升網站的效能

此外，React-Native的推出，讓React架構應用於Android、iOS 和 UPD等程式



15. 專業ReactJS應用I

應用ReactJS

- **React-native**是一個移動應用程式構建框架，採用與**React**相同的設計，讓使用者能夠新增/運用以**React**為核心的移動**UI**組件
- **React**組件是獨立和可重複使用的程式碼
- 可通過服務器端的渲染功能回傳**HTML**



15. 專業ReactJS應用I

應用ReactJS

- 每個React應用程式都會有一個根組件，該組件後面可以接著子組件，形成一個整體的"組件樹"



一些使用React的企業包括：Uber, Airbnb, Facebook, Amazon, Twitter, Netflix等

15. 專業ReactJS應用I

ReactJS的主要特點 - 組件

- 組件是**React**中的建構模組，利用開源的JavaScript庫來打造專為單頁應用程式設計的使用者介面
- 可處理網頁和移動應用程式的視圖層(view layer)
- **React**還允許建立可重複使用的UI組件



15. 專業ReactJS應用I

ReactJS的主要特點 - 組件

- 構建可管理自身狀態的封裝組件，然後將其組合成複雜的UI
- 由於組件邏輯是用JavaScript而非範本撰寫，所以可以很容易地將數據傳遞到應用程式中，並將狀態保持在DOM之外



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

ReactJS的主要特點 - 宣告式

- React使建立互動式介面變得容易
- 為應用程式中的每個狀態設計簡單的視圖，而當數據發生變化時，React將有效地更新和渲染正確的組件
- 宣告式視圖(declarative views)使程式碼更可預測，更容易調試



15. 專業ReactJS應用I

ReactJS的主要特點 - 宣告式

- "宣告式"，至少在編程意義上，通常被定義為"告訴做什麼，而不是怎麼做"
- React是宣告式的，當編寫一個組件程式時，只需告訴React希望DOM是什麼樣子的，然後讓React來處理



15. 專業ReactJS應用I

ReactJS的主要特點 - 宣告式

- 不必擔心當前網頁中的不同元素要如何變化，只需說明"想在那裡有一個按鈕"，"想何時讓這個按鈕啟用或失效"，**React**都可以支援實現
- 使用**React**元素來告訴**React**在渲染每個組件時要調用哪些函式，而庫則決定何時調用
- 這使得組件的渲染是宣告式的



15. 專業ReactJS應用I

ReactJS的主要特點 - 靈活性

- **React**可在任何地方編寫和部署
- 除了技術堆棧的使用，還可以在**React**中開發新功能，而無需重寫現有程式碼
- **React**還可透過**Node**在伺服器上渲染，並使用**React Native**為移動應用程式提供支持



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

15. 專業ReactJS應用I

宣告式渲染(Declarative Rendering)

为了更好地掌握宣告式渲染，
可以參考這個例子

將Button組件作為一個普通的JS
函式 (`<p>{`${theme}`}`
`box`}</p>`) : 傳遞想要使用的
props，並得到理想效果

App.js

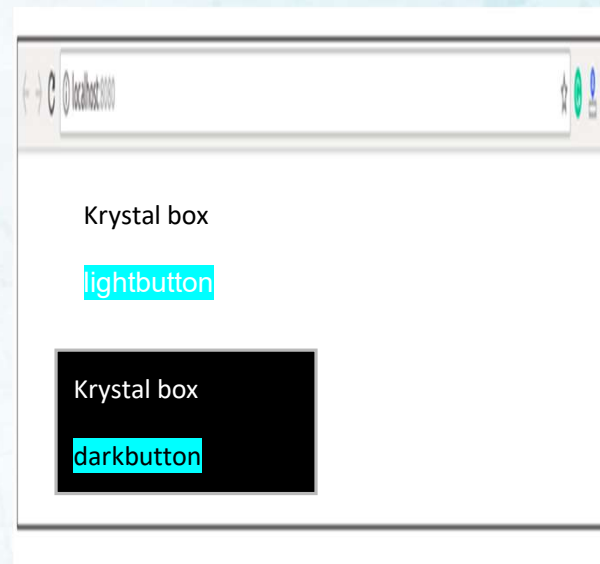
```
import React from "react";
import "./styles.css";

function Button({ text, theme }) {
  return (
    <button className={`Button Button--${theme}`}>
      ({theme}) {text}
    </button>
  );
}

function Krystal({ theme }) {
  return (
    <div className={`Box Box--${theme}`}>
      <p>{`${theme} box`}</p>
      {Button({ text: "button", theme })}
    </div>
  );
}

export default function App() {
  return (
    <>
      <Box theme="light" />
      <Box theme="dark" />
    </>
  );
}
```

輸出



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

15. 專業ReactJS應用I

建立組件

狀態組件

除了獲取輸入數據(通過 **this.props** 存取)之外，組件還可以維護內部狀態 **state** 數據(通過 **this.state** 存取)

當組件的 **state** 數據更改時，渲染的標記將通過重新調用 **render()** 來更新

App.js

```
class Timer extends React.Component {
  constructor(props) {
    super(props);
    this.state = { seconds: 0 };
  }
  tick() {
    this.setState(state => ({
      seconds: state.seconds + 1
    }));
  }
  componentDidMount() {
    this.interval = setInterval(() => this.tick(), 1000);
  }
  componentWillUnmount() {
    clearInterval(this.interval);
  }
  render() {
    return (
      <div>
        Seconds: {this.state.seconds}
      </div>
    );
  }
}
ReactDOM.render(
  <Timer />,
  document.getElementById('timer-example')
);
```

輸出

Second : 1453

15. 專業ReactJS應用I 建立組件

應用程式

使用props和state，可以組合一個小型的Todo應用程式

這個例子利用state追蹤目前的項目清單和使用者的輸入的文字

雖然事件處理器是內聯(inline)渲染的，但實際上會通過事件委派(event delegation)的方式被集中處理

```
class TodoApp extends React.Component {
  constructor(props) {
    super(props);
    this.state = { items: [], text: '' };
    this.handleChange = this.handleChange.bind(this);
    this.handleSubmit = this.handleSubmit.bind(this);
  }

  render() {
    return (
      <div>
        <h3>TODO</h3>
        <TodoList items={this.state.items} />
        <form onSubmit={this.handleSubmit}>
          <label htmlFor="new-todo">
            What needs to be done?
          </label>
          <input
            id="new-todo"
            onChange={this.handleChange}
            value={this.state.text}
          />
          <button>
            Add #({this.state.items.length + 1})
          </button>
        </form>
      </div>
    );
  }

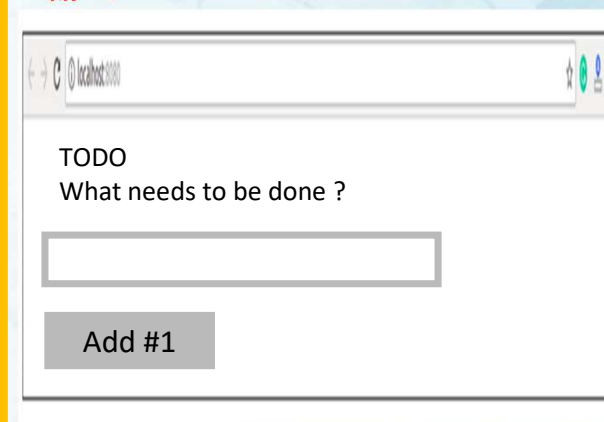
  handleChange(e) {
    this.setState({ text: e.target.value });
  }

  handleSubmit(e) {
    e.preventDefault();
    if (this.state.text.length === 0) {
      return;
    }
    const newItem = {
      text: this.state.text,
      id: Date.now()
    };
    this.setState(state => ({
      items: state.items.concat(newItem),
      text: ''
    }));
  }
}

class TodoList extends React.Component {
  render() {
    return (
      <ul>
        {this.props.items.map(item => (
          <li key={item.id}>{item.text}</li>
        ))}
      </ul>
    );
  }
}

ReactDOM.render(
  <TodoApp />,
  document.getElementById('todos-example')
);
```

輸出



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

15. 專業ReactJS應用I

建立組件

使用外部插件的組件

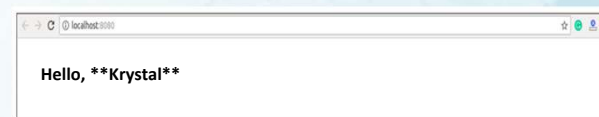
React可與其他函式庫和框架
進行互動

這個例子使用外部Markdown
函式庫`remarkable`，即時轉
換`<textarea>`的值

```
class MarkdownEditor extends React.Component {
  constructor(props) {
    super(props);
    this.md = new Remarkable();
    this.handleChange = this.handleChange.bind(this);
    this.state = { value: 'Hello, **Krystal**' };
  }
  handleChange(e) {
    this.setState({ value: e.target.value });
  }
  getRawMarkup() {
    return { __html: this.md.render(this.state.value) };
  }
  render() {
    return (
      <div className="MarkdownEditor">
        <h3>Input</h3>
        <label htmlFor="markdown-content">
          Enter some markdown
        </label>
        <textarea
          id="markdown-content"
          onChange={this.handleChange}
          defaultValue={this.state.value}
        />
        <h3>Output</h3>
        <div
          className="content"
          dangerouslySetInnerHTML={this.getRawMarkup()}
        />
      </div>
    );
  }
}

ReactDOM.render(
  <MarkdownEditor />,
  document.getElementById('markdown-example')
);
```

輸入



輸出



All Rights Reserved. Reprinting will not be tolerated.
版權所有 · 翻印必究

15. 專業ReactJS應用I

JSX

- JavaScript XML (JSX)是基於ES6的JavaScript的語法擴展，ES6發佈後幾乎被所有主要瀏覽器支援
- JSX允許透過將HTML轉換為React組件來更有效率地為網頁應用程式建立使用者介面



```
class HelloWorld extends React.Component {  
  render() {  
    return (  
      <h1 className='large'>Hello World</h1>  
    );  
  }  
}
```

A screenshot of a code editor with a dark theme. The editor shows a file named 'index.html' with the following code:

```
1 <body>  
2 <div id="root"></div>  
3 <script src="https://unpkg.com/react@16.12.0/umd/react.development.js"></script>  
4 <script src="https://unpkg.com/react-dom@16.12.0/umd/react-dom.development.js"></script>  
5 <script src="https://unpkg.com/@babel/standalone@7.8.3/babel.js"></script>  
6 <script type="text/babel">  
7   function CharacterCount({text}) {  
8     return (  
9       <div>  
10        {'The text "${text}" has '  
11        {text.length ? <strong>{text.length}</strong> : 'No'  
12        {' characters'}  
13      </div>  
14    )  
15  }  
16  
17  const element = (  
18    <>  
19    <CharacterCount text="Hello World" />  
20    <CharacterCount text="" />  
21    </>  
22  )  
23  
24  ReactDOM.render(element, document.getElementById('root'))  
25 </script>  
26 </body>  
27
```

On the right side of the editor, the rendered output is shown in a light box. It contains two lines of text: "The text 'Hello World' has 11 characters" and "The text '' has No characters".

All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

15. 專業ReactJS應用I

JSX

插值 (Interpolation)

- 插值元件(interpolate component)可支援純文字翻譯與React組件(如連結、粗體文字或其他介面元素)混合使用
- 插值元件允許將翻譯字串中的佔位符替換為值或實際的React組件



```
class HelloWorld extends React.Component {  
  render() {  
    return (  
      <h1 className='large'>Hello World</h1>  
    );  
  }  
}
```

```
index.html package.json sandbox.config.json  
1 <body>  
2 <div id="root"></div>  
3 <script src="https://unpkg.com/react@16.12.0/umd/react.development.js"></script>  
4 <script src="https://unpkg.com/react-dom@16.12.0/umd/react-dom.development.js"></script>  
5 <script src="https://unpkg.com/@babel/standalone@7.8.3/babel.js"></script>  
6 <script type="text/babel">  
7   function CharacterCount({text}) {  
8     return (  
9       <div>  
10        {'The text "${text}" has '  
11        {text.length ? <strong>{text.length}</strong> : 'No'  
12        {' characters'}  
13      </div>  
14    )  
15  }  
16  
17  const element = (  
18    <>  
19    <CharacterCount text="Hello World" />  
20    <CharacterCount text="" />  
21    </>  
22  )  
23  
24  ReactDOM.render(element, document.getElementById('root'))  
25 </script>  
26 </body>  
27
```

The text "Hello World" has 11 characters
The text "" has No characters

All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

數字經濟核心科技系列：前端網站開發人員課程

(二) 進階網絡程式設計

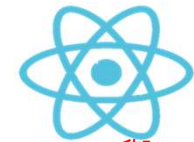
3. React JS

15.2 ReactJS的核心功能

React - 狀態(State)和道具(Props)

React數據通過狀態(State)和道具(Props)函式處理State用於數據管理，而Props用於傳遞數據來自props的數據是唯讀的，不能被外部接收的組件修改State數據可以由其組件修改，但該數據是私有的，不能從外部存取

State和Props的區別在於，State是內部的，由組件本身控制，而Props是外部的，由渲染組件控制



React JS

State與Props的數據

Props

Props是唯讀的

Props不能被修改

States

State變化可以是非同步的

State可以用this.setState來修改狀態

All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

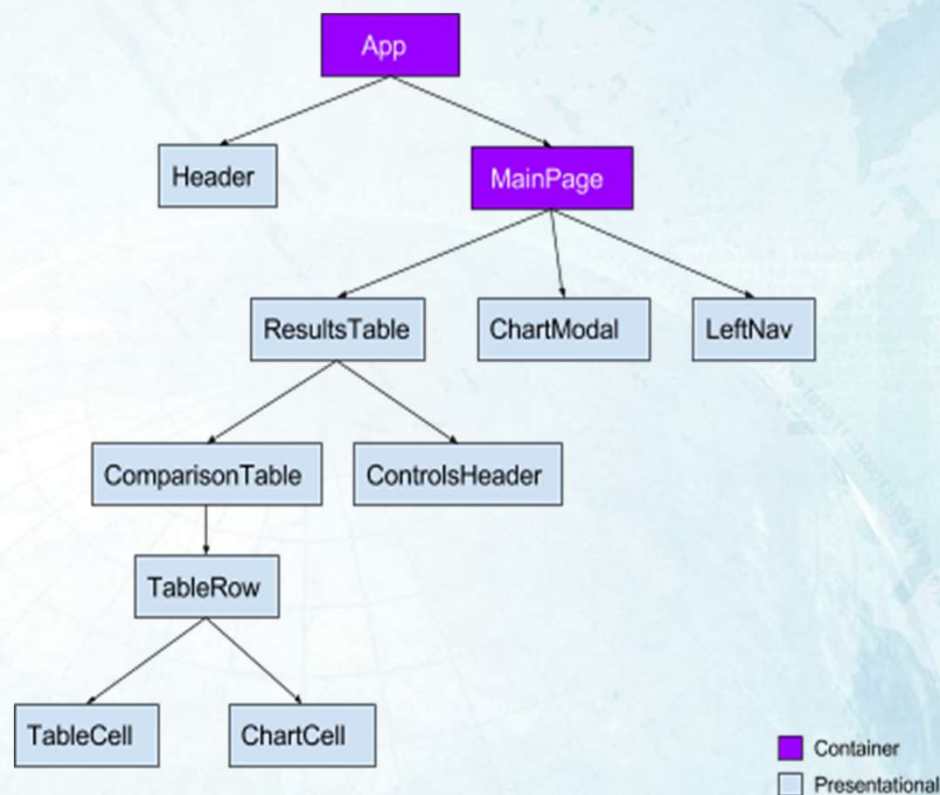
React - Component API

ReactJS組件(Component) API是一個頂級的API，讓程式碼可獨立且重複使用於應用程式中包含多種方法，用於：

- 建立元素
- 轉化元素
- 片段

React組件API中最重要三個方法是：

- `setState()`
- `forceUpdate()`
- `findDOMNode()`



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

15. 專業ReactJS應用I

React - Component API

setState()

這個方法用於更新組件的狀態(state)，但這個方法並不會立即替換state

相反，只會在原來的state上添加變化，作用於回應事件處理程式與伺服器回應時更新使用者介面(UI)

語法

`this.setState(object newState[, function callback]);`

```
import React, { Component } from 'react';
import PropTypes from 'prop-types';
class App extends React.Component {
  constructor() {
    super();
    this.state = {
      msg: "Welcome to Krystal Institute"
    };
    this.updateSetState = this.updateSetState.bind(this);
  }
  updateSetState() {
    this.setState({
      msg: "Krystal Institute ReactJS course"
    });
  }
  render() {
    return (
      <div>
        <h1>{this.state.msg}</h1>
        <button onClick = {this.updateSetState}>SET
STATE</button>
      </div>
    );
  }
}
export default App;
```

Main.js

```
import React from 'react';
import ReactDOM from 'react-dom';
import App from './App.js';

ReactDOM.render(<App/>, document.getElementById('app'));
```

Output



When you click on the **SET STATE** button, you will see the following screen with the updated message.

Output



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

15. 專業ReactJS應用I

React - Component API

forceUpdate()

這個方法支援手動更新該組件

語法

Component.forceUpdate(callback);

App.js

```
import React, { Component } from 'react';
import PropTypes from 'prop-types';
class App extends React.Component {
  constructor() {
    super();
    this.state = {
      msg: "Welcome to Krystal Institute"
    };
    this.updateSetState = this.updateSetState.bind(this);
  }
  updateSetState() {
    this.setState({
      msg: "Krystal Institute ReactJS course"
    });
  }
  render() {
    return (
      <div>
        <h1>{this.state.msg}</h1>
        <button onClick = {this.updateSetState}>SET
STATE</button>
      </div>
    );
  }
}
export default App;
```

輸出



每次點擊**ForceUpdate**按鈕時，將生成隨機數，並顯示在下面的圖片中



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

15. 專業ReactJS應用I

React - Component API

findDOMNode()

對於DOM操作，需要使用
ReactDOM.findDOMNode()
方法，這個方法支援找到
或存取底層DOM節點

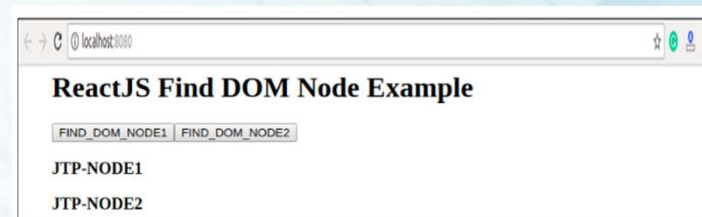
語法

ReactDOM.findDOMNode(
component);

App.js

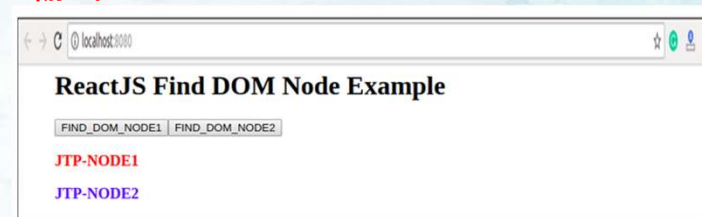
```
1. import React, { Component } from 'react';
2. import ReactDOM from 'react-dom';
3. class App extends React.Component {
4.   constructor() {
5.     super();
6.     this.findDOMNodeHandler1 = this.findDOMNodeHandler1.b
ind(this);
7.     this.findDOMNodeHandler2 = this.findDOMNodeHandler2.b
ind(this);
8.   };
9.   findDOMNodeHandler1() {
10.    var myDiv = document.getElementById('myDivOne');
11.    ReactDOM.findDOMNode(myDivOne).style.color = 'red';
12.  }
13.   findDOMNodeHandler2() {
14.    var myDiv = document.getElementById('myDivTwo');
15.    ReactDOM.findDOMNode(myDivTwo).style.color = 'blue'
;
16.  }
17.   render() {
18.     return (
19.       <div>
20.         <h1>ReactJS Find DOM Node Example</h1>
21.         <button onClick = {this.findDOMNodeHandler1}>FIN
D_DOM_NODE1</button>
22.         <button onClick = {this.findDOMNodeHandler2}>FIN
D_DOM_NODE2</button>
23.         <h3 id = "myDivOne">JTP-NODE1</h3>
24.         <h3 id = "myDivTwo">JTP-NODE2</h3>
25.       </div>
26.     );
27.   }
28. }
29. export default App;
```

輸出



每次點擊這個**按鈕**，節點的顏色就會改變，
並顯示在下面的屏幕上

輸出



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

數字經濟核心科技系列：前端網站開發人員課程

(二) 進階網絡程式設計

3. React JS

15.3 用於ReactJS的AJAX

15. 專業ReactJS應用I

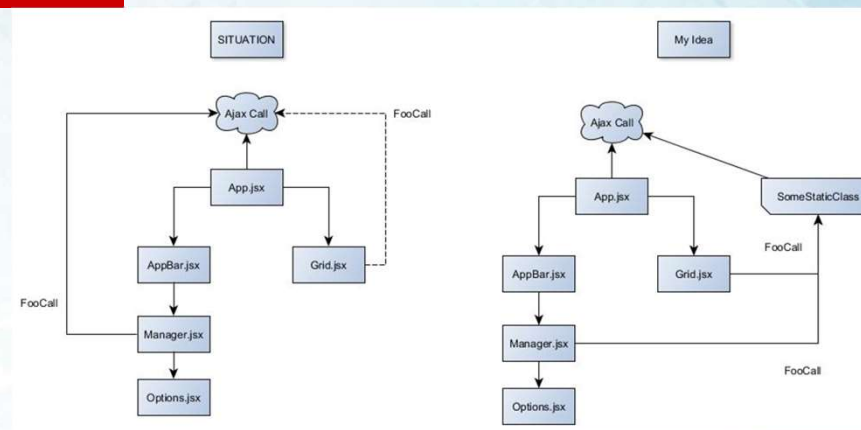
React - AJAX

AJAX是一種用於客戶端的網絡技術，用於執行非同步API調用或HTTP請求。AJAX是指在後台加載數據並顯示在網頁上，而無需重新加載整個頁面。

AJAX=Asynchronous JavaScript and XML

React和AJAX

React一直是關於渲染組件的，它從兩個來源獲取數據，這兩個來源是**props**和**state**，通過這些庫/框架，默認情況下有其獲取數據的方式(或使用AJAX)



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

React - AJAX

AJAX是指一組相互關聯的技術，如JavaScript、CSS(層疊樣式表)、HTML/XHTML、DOM(文件對像模型)、XML(可擴展標記語言)、XMLHttpRequest和Microsoft Object

AJAX開發技術有助於建立更快、更好且高度互動的網站與網頁應用程式

AJAX的核心功能是非同步更新網頁內容，這表示使用者的瀏覽器不需要在改變頁面上部分內容的同時重新載入頁面或回傳

AJAX只允許重要的數據從用戶端傳送到伺服器端

15. 專業ReactJS應用I

React - AJAX

AJAX的主要目標是提高網頁應用程式的速度、可用性和效能

AJAX的主要功能之一是名為**jQuery**的JavaScript object庫，它是JavaScript的包裝器

jQuery是免費的開源軟件，支援容易地編寫客戶端JavaScript來導航和操作頁面，並使用**RxJS** (Reactive Extensions for JavaScript)進行非同步AJAX回調

通過採用JSON (JavaScript Object Notation)和jQuery，AJAX回調已經發展成為設計和開發網絡應用程式的標準編程規模

一些使用AJAX的頂級巨頭：--《衛報》、Netflix、RoundCube、谷歌、CubeSmart等

All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

15. 專業ReactJS應用I

React - Query

jQuery提供多種AJAX功能方法，可以使用HTTP Get和HTTP Post從遠端伺服器請求文本、HTML、XML或JSON - 將外部數據直接載入到網頁選定的HTML元素中

jQuery AJAX

jQuery - AJAX介紹

jQuery - AJAX Load()方法

jQuery - AJAX get() and post()方法

https://www.w3schools.com/jquery/jquery_ajax_get_post.asp

<https://www.tutorialsteacher.com/jquery>

<https://www.tutorialsteacher.com/jquery/jquery-ajax-introduction>

All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

15. 專業ReactJS應用I

React - jQuery

- jQuery是一個小型、快速和輕量級的JavaScript庫
- jQuery是獨立於平台的
- jQuery意味著"少寫多做"
- jQuery簡化了AJAX調用和DOM操作



jQuery的特點

HTML操作

DOM操作

DOM元素選擇

CSS操作

效果和動畫

實用工具

AJAX

HTML事件方法

JSON解析

透過外掛程式的可擴展性

15. 專業ReactJS應用I

React - jQuery

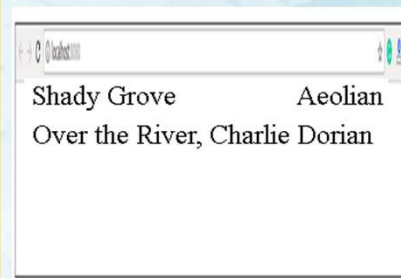
jQuery會與DOM元素互動並執行操作

因此，需要檢測DOM(文件對像模型)何時被完全加載，以便jQuery程式碼開始與DOM互動而不出現問題

jQuery

```
1. <TABLE>
2. <ROWS>
3. <TR>
4. <TD>Shady Grove</TD>
5. <TD>Aeolian</TD>
6. </TR>
7. <TR>
8. <TD>Over the River, Charlie</TD>
9. <TD>Dorian</TD>
10. </TR>
11. </ROWS>
12. </TABLE>
```

輸出



Shady Grove	Aeolian
Over the River, Charlie	Dorian

15. 專業ReactJS應用I

React - jQuery

例如，在網頁中的div標籤上寫"Hello World！"

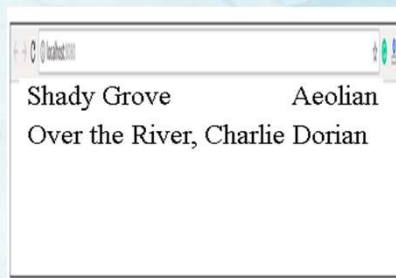
第一步是檢查DOM是否被完全加載，這樣才能確保可以找到DIV元素並寫上"Hello World！"

如果在DOM加載完成之前寫，jQuery可能找不到div元素，而出現錯誤

jQuery

```
1. <TABLE>
2. <ROWS>
3. <TR>
4. <TD>Shady Grove</TD>
5. <TD>Aeolian</TD>
6. </TR>
7. <TR>
8. <TD>Over the River, Charlie</TD>
9. <TD>Dorian</TD>
10. </TR>
11. </ROWS>
12. </TABLE>
```

輸出



Shady Grove	Aeolian
Over the River, Charlie	Dorian

15. 專業ReactJS應用I

AJAX - jQuery

jQuery

API內置函式`ready()`，可偵測特定元素是否準備好(加載)

在這裡，可使用`ready()`來檢查整個文件對像是否已被加載，這樣便可以確定所有DOM元素也已成功加載

如圖所示，文件對像在瀏覽器中是全域物件，因此也可以寫入
`window.document`查找文件對象

jQuery

```
1.<!DOCTYPE html>
2.<html>
3.<head>
4.<script
src="https://ajax.googleapis.com/ajax/libs/jq
uery/
3.5.1/jquery.min.js"></script>
5.<script>
6.$(document).ready(function(){
7.$("button").click(function(){
8.$("#div1").load("demo_test.txt");
9.});
10.});
11.</script>
12.</head>
13.<body>
14.<div id="div1"><h2>Let jQuery AJAX
Change
This Text</h2></div>
15.<button>Get External Content</button>
16.</body>
17.</html>
```

輸出



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

15. 專業ReactJS應用I

AJAX - jQuery示例

要建立第一個jQuery檔，需要有可以調用jQuery的JavaScript檔

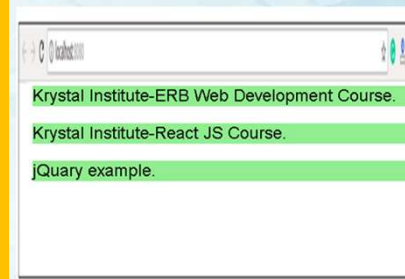
可以從jquery.com下載jQuery檔，或者透過jQuery檔的絕對網址(URL)直接調用

在這個jQuery示例中，使用了jQuery檔的絕對URL

jQuery

```
1.<!DOCTYPE html>
2.<html>
3.<head>
4.<title>First jQuery Example</title>
5.<script type="text/javascript"
src="https://ajax.googleapis.com/ajax/libs/jquery/
2.1.3/jquery.min.js">
6.</script>
7.<script type="text/javascript"
language="javascript">
8.$(document).ready(function() {
9.$("p").css("background-color", "lightgreen");
10.});
11.</script>
12.</head>
13.<body>
14.<p>Krystal Institute-ERB Web Development
Course.</p>
15.<p>Krystal Institute-React JS Course.</p>
16.<p>jQuery example.</p>
17.</body>
18.</html>
```

輸出



15. 專業ReactJS應用I

AJAX - jQuery示例

要建立第一個jQuery檔，需要有可以調用jQuery的JavaScript檔

可以從jquery.com下載jQuery檔，或者透過jQuery檔的絕對URL直接調用

在這個jQuery示例中，使用了jQuery檔的絕對URL

練習：

更改顏色背景

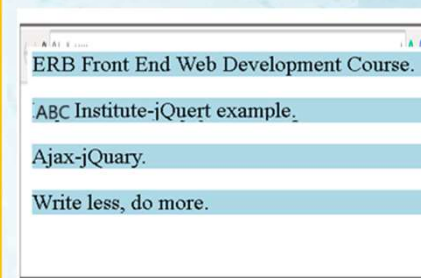
插入新名稱

使用新名稱添加新行

jQuery

```
1.<!DOCTYPE html>
2.<html>
3.<head>
4. <title>First jQuery Example</title>
5. <script type="text/javascript"
  src="https://ajax.googleapis.com/ajax/libs/jquery/
  2.1.3/jquery.min.js">
6. </script>
7. <script type="text/javascript"
  language="javascript">
8. $(document).ready(function() {
9. $("p").css("background-color", "lightblue");
10. });
11. </script>
12. </head>
13. <body>
14. <p>ERB Front End Web Development
  Course.</p>
15. <p>Krystal Institute-jQuert example.</p>
16. <p>Ajax-jQuary.</p>
17. <p>Write less, do more.</p>
18. </body>
19. </html>
```

輸出



AJAX - jQuery示例

jQuery的attr()方法用於獲取或設置DOM元素指定屬性的值

在這個例子中，`$('#p').attr('style')`會獲取HTML頁面中第一個<p>元素的style屬性，但不會返回其他<p>元素的style屬性

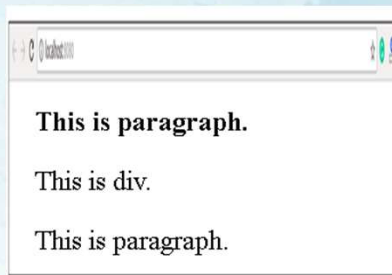
語法

`$('#selector expression').attr('name','value');`

jQuery attr() 方法

```
1.$('p').attr('style'); // returns "font-size:16px;fontweight:
bold"
2.$('div').attr('class','yellowDiv'); // adds
class='divCls' to each div element
3.<div>
4. This is div.
5.</div>
6.<p style="font-size:16px;font-weight:bold">
7. This is paragraph.
8.</p>
9.<div>
10. This is div.
11.</div>
12.<p>
13. This is paragraph.
14.</p>
```

輸出



All Rights Reserved. Reprinting will not be tolerated.
版權所有 · 翻印必究

15. 專業ReactJS應用I

AJAX - jQuery示例

要建立第一個jQuery檔，需要有可以調用jQuery的JavaScript檔

在這個jQuery示例中，使用了jQuery檔的絕對URL

練習：

添加顏色背景

添加新名稱 (ABC Web Development)

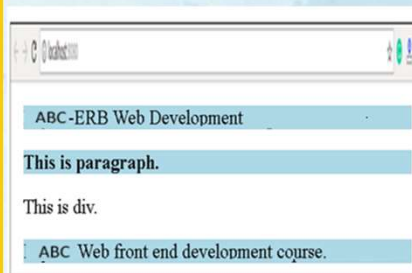
添加新行和名稱 (ABC Web Front End Development Course)

更改字體大小

jQuery attr() method

```
$( 'p' ).attr( 'style' ); // returns
"font-size:16px;font-weight:
bold"
$( 'div' ).attr( 'class','yellowDiv' ); // adds class='divCls' to
each div element
<script type="text/javascript"
src="https://ajax.googleapis.com/ajax/libs/jquery/2.1.
3/jqu
ery.min.js">
</script>
<script type="text/javascript" language="javascript">
$(document).ready(function() {
$( "p" ).css( "background-color", "lightblue" );
});
</script>
</head>
<body>
<p>ABC Web Development
</div>
<p style="font-size:16px;font-weight:bold">
This is paragraph.
</p>
<div>
This is div.
</div>
<p>
ABC Web Front End Development Course
</p>
```

輸出



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

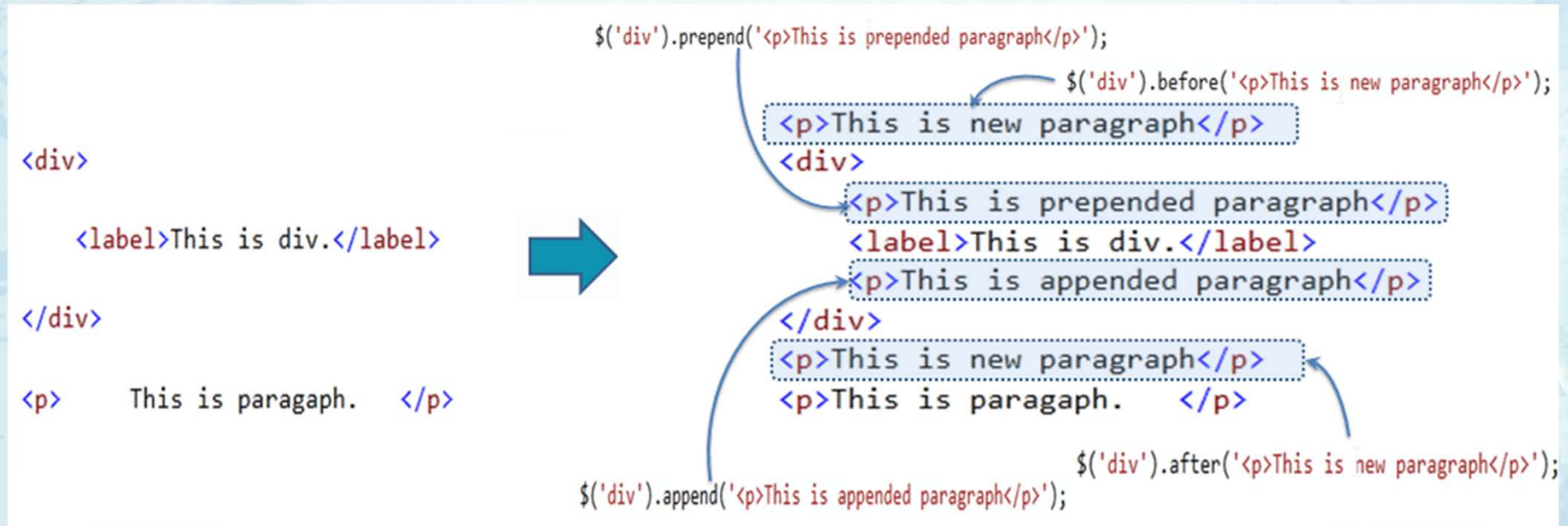
AJAX - DOM操作示例

下表列出了一些添加/刪除新DOM元素的重要方法

方法	說明
append()	將內容插入到由選擇器指定的元素的末尾
before()	在選擇器指定的元素之前插入內容(新的或現有的DOM元素)
after()	在選擇器指定的元素之後插入內容(新的或現有的DOM元素)
prepend()	在選擇器指定的元素的開頭插入內容
remove()	從DOM中刪除由選擇器指定的元素
replaceAll()	將目標元素替換為指定的元素
wrap()	將HTML結構包裝在選擇器指定的每個元素周圍

15. 專業ReactJS應用I

AJAX - DOM操作示例



15. 專業ReactJS應用I

AJAX - jQuery示例

jQuery

```
<!DOCTYPE html>
<html>
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>
<script>
$(document).ready(function(){
$("#myInput").on("keyup", function() {
var value = $(this).val().toLowerCase();
$("#myTable tr").filter(function() {
$(this).toggle($(this).text().toLowerCase().indexOf(value) > -1)
});
});
});
</script>
<style>
table {
font-family: arial, sans-serif;
border-collapse: collapse;
width: 100%;
}
td, th {
border: 1px solid #dddddd;
text-align: left;
padding: 8px;
}
tr:nth-child(even) {
background-color: #dddddd;
}
</style>
</head>
<body>
<h2>Filterable Table</h2>
<p>Type something in the input field to search the table for first names, last names
or emails:</p>
<input id="myInput" type="text" placeholder="Search..">
<br><br>
```

```
<table>
<thead>
<tr>
<th>Firstname</th>
<th>Lastname</th>
<th>Email</th>
</tr>
</thead>
<tbody id="myTable">
<tr>
<td>John</td>
<td>Doe</td>
<td>john@example.com</td>
</tr>
<tr>
<td>Mary</td>
<td>Moe</td>
<td>mary@mail.com</td>
</tr>
<tr>
<td>July</td>
<td>Dooley</td>
<td>july@greatstuff.com</td>
</tr>
<tr>
<td>Anja</td>
<td>Ravendale</td>
<td>a_r@test.com</td>
</tr>
</tbody>
</table>
<p>Note that we start the search in tbody, to prevent filtering the table
headers.</p>
</body>
</html>
```

輸出

Filterable Table

Type something in the input field to search the table for first names, last names or emails:

Firstname	Lastname	Email
John	Doe	john@example.com
Mary	Moe	mary@mail.com
July	Dooley	july@greatstuff.com
Anja	Ravendale	a_r@test.com

Note that we start the search in tbody, to prevent filtering the table headers.

15. 專業ReactJS應用I

參考

<https://technostacks.com/blog/best-frontend-frameworks/>

<https://www.sam-solutions.com/blog/best-frontend-framework/>

<https://www.simform.com/blog/best-frontend-frameworks/>

<https://dev.to/ericchapman/your-first-steps-with-express-js-34op>

[https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Client-side JavaScript frameworks/Vue getting started](https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Client-side_JavaScript_frameworks/Vue_getting_started)